On this page

Storefront MCP s

Copy page MD

Connect your AI agent to a specific Shopify store's catalog, shopping cart, and policies. The Storefront MCP server helps customers browse and buy with their selected merchant.

How it worl

The Storefront MCP server lets your AI agent handle shopping tasks for the selected store:

1. A shopper asks about products while browsing a store.
2. Your agent searches the store's catalog and manages carts.
3. The shopper adds items and completes checkout.

Connect to the ser

Each Shopify store has its own MCP server endpoint that exposes storefront features. This endpoint handles all server calls for product search, cart operations, and policy questions:

```
https://storedomain.com/api/mcp
```

This endpoint is unique to each store and gives access to all storefront commerce capabilities. Your app needs to configure this endpoint based on which store the customer is shopping with.

Caution

By using the Shopify MCP servers, you agree to the Shopify API License and Terms of Use .

Create an API request to the Storefront MC

Storefront MCP servers don't require authentication:

1. Replace `storedomain` with the store's actual domain.
2. Send requests to the store's MCP endpoint.
3. Include the Content-Type header.

Here's how to set up a request:

```
// Basic setup for Storefront MCP server requests
const storeDomain = 'your-store.myshopify.com';
const mcpEndpoint = `https://${storeDomain}/api/mcp`;

// Example request using the endpoint
fetch(mcpEndpoint, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    jsonrpc: '2.0',
    method: 'tools/call',
    id: 1,
    params: {
      name: 'search_shop_catalog',
      arguments: { query: 'coffee', context: "reader" }
    }
  })
});
```

Note

Note: Some stores may restrict access. Always test with your specific store.

Available toc

The Storefront MCP server provides a set of tools to help customers browse and buy from a specific store. Use the `tools/list` command to discover available tools and their capabilities. Each tool is documented with a complete schema that defines its parameters, requirements, and response format.

search_shop_catalog

Searches the store's product catalog to find items that match your customer's needs.

Key parameters:

- `query`: The search query to find related products (required)
- `context`: Additional information to help tailor results (required)

Response includes:

- Product name, price, and currency
- Variant ID
- Product URL and image URL
- Product description

Example use:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 1,
  "params": {
    "name": "search_shop_catalog",
    "arguments": {
      "query": "organic coffee beans",
      "context": "Customer prefers fair trade products"
    }
  }
}
```

Note

Tip: Create Markdown links for product titles using the URL property to help customers navigate to product pages.

search_shop_policies_and_faqs

Answers questions about the store's policies, products, and services to build customer trust.

Key parameters:

- `query`: The question about policies or FAQs (required)
- `context`: Additional context like current product (optional)

Example use:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 1,
  "params": {
    "name": "search_shop_policies_and_faqs",
    "arguments": {
      "query": "What is your return policy for sale items?",
      "context": "Customer is looking at discounted winter jackets"
    }
  }
}
```

Note

Tip: Use only the provided answer to form your response. Don't include external information that might be inaccurate. For better store policy management, consider using the Knowledge Base app to configure store policies.

get_cart

Retrieves the current contents of a cart, including item details and checkout URL.

Key parameters:

- `cart_id`: ID of an existing cart

Example use:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 1,
  "params": {
    "name": "get_cart",
    "arguments": {
      "cart_id": "gid://shopify/Cart/abc123def456"
    }
  }
}
```

update_cart

Updates quantities of items in an existing cart or adds new items. Creates a new cart if

no cart ID is provided. Set quantity to 0 to remove an item.

Key parameters:

- `cart_id`: ID of the cart to update. Creates a new cart if not provided.
- `lines`: Array of items to update or add (required, each with `quantity` and optional `line_item_id` for existing items)

Example use:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 1,
  "params": {
    "name": "update_cart",
    "arguments": {
      "cart_id": "gid://shopify/Cart/abc123def456",
      "lines": [
        {
          "line_item_id": "gid://shopify/CartLine/line2",
          "merchandise_id": "gid://shopify/ProductVariant/789012",
          "quantity": 2
        }
      ]
    }
  }
}
```

Was this page helpful? Yes No

Updates

Developer changelog

Shopify Editions

Business growth

Shopify Partners Program

Shopify App Store

Shopify Academy

Legal

Terms of service

API terms of use

Privacy policy

Partners Program Agreement

Shopify

About Shopify

Shopify Plus

Careers

Investors

Press and media